

Tema 4 — Clases y objetos

Laboratorio Guiado

Objetivos	Practicar la creación de clases y objetos en Python: definición de clases, instancias, atributos, métodos, <code>__init__()</code> , <code>self</code> , <code>__str__()</code> , atributos de clase, encapsulación, propiedades, composición, herencia, sobrescritura y polimorfismo básico.
Material necesario	Terminal, Python 3.13, venv del Tema04, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en <code>~/Curso_Python/Tema04</code> .

Laboratorio 1. Clase, objeto e instancia

1. Actualizar el repositorio y abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema04
source .venv/bin/activate
python --version
code .
```

2. Crear o abrir el fichero `01_clase_objeto_instancia.py`. Definir una clase mínima usando `pass` para mantener el bloque vacío de forma válida.

`pass` no ejecuta ninguna acción. Solo indica a Python que el bloque existe aunque todavía no tenga implementación.

```
class Servicio:
    pass

ssh = Servicio()
web = Servicio()

print(type(ssh))
print(type(web))
print(ssh)
print(web)
print(ssh is web)
```

3. Ejecutar el script y comprobar que cada llamada a la clase crea un objeto independiente.

```
python 01_clase_objeto_instancia.py
```

Resultado esperado: se muestra que `ssh` es una instancia de `Servicio` y que `ssh` y `web` no son el mismo objeto.

Laboratorio 2. Atributos, métodos y `self`

1. Crear o abrir el fichero `02_atributos_metodos_self.py`. Definir una clase `Servicio` con atributos de instancia inicializados desde `__init__()`.

`__init__()` se ejecuta al crear la instancia. `self` representa el objeto concreto que se está inicializando.

```
class Servicio:
    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

    def esta_activo(self):
        return self.activo

ssh = Servicio("ssh")
httpd = Servicio("httpd", True)

print(ssh.nombre, ssh.esta_activo())
print(httpd.nombre, httpd.esta_activo())
```

2. Agregar un método que modifique el estado interno del objeto y comprobar que solo afecta a esa instancia.

```
class Servicio:
    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

    def esta_activo(self):
        return self.activo

    def iniciar(self):
        self.activo = True

    def detener(self):
        self.activo = False

ssh = Servicio("ssh")
httpd = Servicio("httpd", True)

ssh.iniciar()
print(ssh.nombre, ssh.esta_activo())
print(httpd.nombre, httpd.esta_activo())
```

3. Ejecutar el fichero y revisar que los atributos `nombre` y `activo` pertenecen a cada objeto.

```
python 02_atributos_metodos_self.py
```

Resultado esperado: el alumno debe identificar que `self` permite acceder al estado propio de cada instancia.

Laboratorio 3. Representación textual y atributos de clase

1. Crear o abrir el fichero 03_str_y_atributos_clase.py. Imprimir una instancia sin `__str__()`. Ejecútalo para observar la representación técnica por defecto (Indica la posición de memoria).

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

MiServicio1 = Servicio("web", 80)
print(MiServicio1)
print(f"Servicio registrado -> {MiServicio1.nombre} -> {MiServicio1.puerto}")
```

2. Añadir `__str__()` para devolver una cadena legible. El método no debe imprimir: debe devolver texto con `return`. Observa cómo ahora al imprimir directamente el servicio devuelve el método `__str__()`.

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    def __str__(self):
        return f"{self.nombre}:{self.puerto}"

MiServicio1 = Servicio("web", 80)
print(MiServicio1)
print(f"Servicio registrado -> {MiServicio1.nombre} -> {MiServicio1.puerto}")
```

3. Revisar la diferencia entre un atributo de clase y un atributo de instancia.

Un atributo de clase se comparte por todas las instancias. Un atributo de instancia pertenece a cada objeto concreto. Si cambias el atributo para una instancia concreta, no cambia para las otras.

```
class Servicio:
    categoria = "Sistema"

    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    def __str__(self):
        return f"{self.nombre}:{self.puerto}"

MiServicio1 = Servicio("web", 80)
print(MiServicio1)
print(f"Servicio registrado -> {MiServicio1.nombre} -> {MiServicio1.puerto}")

MiServicio2 = Servicio("ssh", 22)
print(MiServicio2)
print(f"Servicio registrado -> {MiServicio2.nombre} -> {MiServicio2.puerto}")
print()
print(MiServicio1.categoria, MiServicio2.categoria)
print("Cambiamos categoria en MiServicio2")
MiServicio1.categoria = "aplicacion"
print(MiServicio1.categoria, MiServicio2.categoria)
```

4. Ejecutar el script completo.

```
python 03_str_y_atributos_clase.py
```

Resultado esperado: `__str__()` mejora la salida por pantalla y los atributos de instancia pueden diferir entre objetos.

Laboratorio 4. Encapsulación básica y propiedades

1. Crear o abrir el fichero `04_encapsulacion_propiedades_a.py`. Empezar con una clase que usa una convención de uso interno con `_activo` y un atributo con name mangling `__puerto`.

Python no impone privacidad absoluta. El guion bajo inicial indica uso interno por convención. El doble guion bajo aplica name mangling y dificulta el acceso directo. Nota: Si descomentas la última línea recibes `AttributeError`

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self._activo = False
        self.__puerto = puerto

    def iniciar(self):
        self._activo = True

    def get_activo(self):
        return self._activo

    def get_puerto(self):
        return self.__puerto

ssh = Servicio("ssh", 22)
ssh.iniciar()
print(ssh.nombre)
print(ssh.get_activo())
print(ssh.get_puerto())
print(ssh._activo)
# print(ssh.__puerto) # AttributeError
```

2. Ejecutar el fichero y comprobar que el setter de nombre elimina espacios y convierte a minúsculas.

```
python 04_encapsulacion_propiedades_a.py
```

3. Crear o abrir una versión 04_encapsulacion_propiedades_b.py con @property y setter para normalizar el nombre del servicio y validar el puerto.

```
class Servicio:
    def __init__(self, nombre, puerto):
        self.nombre = nombre
        self.puerto = puerto

    @property
    def nombre(self):
        return self._nombre

    @nombre.setter
    def nombre(self, valor):
        self._nombre = valor.strip().lower()

    @property
    def puerto(self):
        return self._puerto

    @puerto.setter
    def puerto(self, valor):
        if not 1 <= valor <= 65535:
            raise ValueError("Puerto no válido")
        self._puerto = valor

ssh = Servicio(" SSH ", 22)
print(ssh.nombre, ssh.puerto)

# Descomenta para comprobar la validación:
#ssh.puerto = 70000
```

4. Ejecutar el fichero y comprobar que el setter de nombre elimina espacios y convierte a minúsculas.

```
python 04_encapsulacion_propiedades_b.py
```

Resultado esperado: el alumno debe entender que @property permite leer como atributo, pero ejecutar lógica interna de validación o normalización.

Laboratorio 5. Composición: objetos dentro de objetos

1. Crear o abrir el fichero 05_composicion.py. Definir una clase InterfazRed y una clase Servidor que contiene una interfaz como atributo.

La composición expresa una relación “tiene un”. Un servidor tiene una interfaz de red; no es una interfaz de red.

```
class InterfazRed:
    def __init__(self, nombre, ip):
        self.nombre = nombre
        self.ip = ip

    def __str__(self):
        return f"{self.nombre} ({self.ip})"

class Servidor:
    def __init__(self, hostname, interfaz):
        self.hostname = hostname
        self.interfaz = interfaz

    def resumen_red(self):
        return f"{self.hostname} -> {self.interfaz}"

eth0 = InterfazRed("eth0", "10.0.0.20")
srv = Servidor("srv-web-01", eth0)

print(srv.resumen_red())
```

2. Modificar la interfaz del servidor creando un nuevo objeto InterfazRed y asignándolo al atributo interfaz.

```
srv.interfaz = InterfazRed("eth1", "10.0.1.20")
print(srv.resumen_red())
```

3. Agregar una clase ConfiguracionServicio y asociarla a un servicio para practicar otra composición sencilla.

```
class ConfiguracionServicio:
    def __init__(self, puerto, protocolo):
        self.puerto = puerto
        self.protocolo = protocolo

    def __str__(self):
        return f"{self.protocolo}/{self.puerto}"

class Servicio:
    def __init__(self, nombre, configuracion):
        self.nombre = nombre
        self.configuracion = configuracion

    def resumen(self):
        return f"{self.nombre} -> {self.configuracion}"

web = Servicio("nginx", ConfiguracionServicio(80, "tcp"))
print(web.resumen())
```

4. Ejecutar el fichero completo.

```
python 05_composicion.py
```

Resultado esperado: se debe ver que una clase puede construir objetos más complejos usando otros objetos como atributos.

Laboratorio 6. Herencia, super(), sobrescritura y polimorfismo

1. Crear o abrir el fichero 06_herencia_polimorfismo.py. Definir una clase base Servicio y una clase hija ServicioWeb.

La herencia expresa una relación “es un tipo de”. ServicioWeb es un tipo especializado de Servicio.

```
class Servicio:
    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

    def descripcion(self):
        return f"{self.nombre}: servicio genérico"

class ServicioWeb(Servicio):
    def __init__(self, nombre, puerto, activo=False):
        super().__init__(nombre, activo)
        self.puerto = puerto

    def descripcion(self):
        estado = "activo" if self.activo else "detenido"
        return f"{self.nombre}:{self.puerto} -> {estado}"

web1 = ServicioWeb("nginx", 80, True)
web2 = ServicioWeb("httpd", 8080)

print(web1.descripcion())
print(web2.descripcion())
```

2. Agregar otra clase con el mismo método descripcion() para comprobar polimorfismo básico.

```
class ServicioBD(Servicio):
    def __init__(self, nombre, motor, activo=False):
        super().__init__(nombre, activo)
        self.motor = motor

    def descripcion(self):
        return f"{self.nombre} -> motor {self.motor}"

servicios = [ServicioWeb("https", 443, True), ServicioBD("base-datos",
"PostgreSQL", True)]

for servicio in servicios:
    print(servicio.descripcion())
```

3. Revisar un ejemplo breve de herencia múltiple. Se muestra para reconocerla, no para usarla como primera opción de diseño.

```
class Monitorizable:
    def comprobar(self):
        return "comprobado"

class Reinicializable:
    def reiniciar(self):
        return "reiniciado"

class ServicioSistema(Monitorizable, Reinicializable):
    pass

svc = ServicioSistema()
print(svc.comprobar())
print(svc.reiniciar())
```

4. Ejecutar el script completo.

```
python 06_herencia_polimorfismo.py
```

Resultado esperado: la clase hija reutiliza la inicialización de la clase base, sobrescribe métodos y puede tratarse de forma común cuando ofrece la misma operación.

Laboratorio 7. Composición frente a herencia

1. Crear o abrir el fichero 07_composicion_vs_herencia.py. Comparar una relación de herencia con una relación de composición.

```
class Servicio:
    pass

# Herencia: ServicioWeb es un tipo de Servicio
class ServicioWeb(Servicio):
    pass

# Composición: Servidor tiene servicios
class Servidor:
    def __init__(self, nombre, servicio_principal):
        self.nombre = nombre
        self.servicio_principal = servicio_principal

web = ServicioWeb()
srv = Servidor("srv-web-01", web)

print(type(web))
print(type(srv.servicio_principal))
print(type(srv))
```


2. Agregar un ejemplo donde la composición resulta más clara que la herencia: una tarea programada usa una alerta, pero no es una alerta.

```
class Alerta:
    def __init__(self, canal):
        self.canal = canal

    def enviar(self, mensaje):
        return f"[{self.canal}] {mensaje}"

class TareaProgramada:
    def __init__(self, nombre, hora, alerta):
        self.nombre = nombre
        self.hora = hora
        self.alerta = alerta

    def ejecutar(self):
        mensaje = f"Tarea {self.nombre} ejecutada a las {self.hora}"
        return self.alerta.enviar(mensaje)

tarea = TareaProgramada("backup", "02:00", Alerta("correo"))
print(tarea.ejecutar())
```

3. Ejecutar el fichero completo y revisar qué relación se entiende mejor en cada caso.

```
python 07_composicion_vs_herencia.py
```

Resultado esperado: la herencia sirve para especializar; la composición sirve para construir objetos combinando piezas.

Laboratorio 8. Ejemplo integrado: servidor, interfaz y servicios

1. Crear o abrir el fichero 08_ejemplo_integrado.py. Este ejercicio integra los conceptos principales del tema en un ejemplo único.

El ejemplo usa clases, instancias, atributos, métodos, `__str__()`, propiedades, composición, herencia, sobrescritura y polimorfismo básico. Las listas de servicios se usarán de forma sencilla; se estudiarán con más detalle en el tema de colecciones.

```
class InterfazRed:
    """Representa una interfaz de red de un servidor."""

    def __init__(self, nombre, ip):
        self.nombre = nombre
        self.ip = ip

    def __str__(self):
        return f"{self.nombre} ({self.ip})"

class Servicio:
    """Clase base para servicios del sistema."""

    categoria = "servicio"

    def __init__(self, nombre, activo=False):
        self.nombre = nombre
        self.activo = activo

    @property
    def nombre(self):
        return self._nombre
```

```
@nombre.setter
def nombre(self, valor):
    self._nombre = valor.strip().lower()

def iniciar(self):
    self.activo = True

def comprobar(self):
    estado = "OK" if self.activo else "DETENIDO"
    return f"{self.nombre}: {estado}"

def __str__(self):
    return self.comprobar()

class ServicioWeb(Servicio):
    """Servicio especializado con un puerto TCP asociado."""

    def __init__(self, nombre, puerto, activo=False):
        super().__init__(nombre, activo)
        self.puerto = puerto

    @property
    def puerto(self):
        return self._puerto

    @puerto.setter
    def puerto(self, valor):
        if not 1 <= valor <= 65535:
            raise ValueError("Puerto no válido")
        self._puerto = valor

    def comprobar(self):
        estado = "OK" if self.activo else "DETENIDO"
        return f"{self.nombre}:{self.puerto} -> {estado}"

class Servidor:
    """Servidor compuesto por una interfaz de red y varios servicios."""

    def __init__(self, hostname, interfaz):
        self.hostname = hostname
        self.interfaz = interfaz
        self.servicios = []

    def agregar_servicio(self, servicio):
        self.servicios.append(servicio)

    def resumen(self):
        print(f"Servidor: {self.hostname}")
        print(f"Red: {self.interfaz}")
        for servicio in self.servicios:
            print("-", servicio.comprobar())

interfaz = InterfazRed("eth0", "10.0.0.20")
servidor = Servidor("srv-web-01", interfaz)

ssh = Servicio(" SSH ", True)
web = ServicioWeb("nginx", 443, True)
api = ServicioWeb("api", 8080, False)

servidor.agregar_servicio(ssh)
servidor.agregar_servicio(web)
servidor.agregar_servicio(api)
```

```
servidor.resumen()
```

2. Ejecutar el script completo y revisar la salida. Después modificar el estado de api a activo y volver a ejecutar.

```
python 08_ejemplo_integrado.py

# Modificación sugerida dentro del script:
# api.iniciar()
# servidor.resumen()
```

Resultado esperado: el alumno debe ver un modelo completo y coherente donde Servidor contiene una interfaz y varios servicios, mientras ServicioWeb especializa Servicio.

Ejercicios propuestos

1. Crear un script src/inventario_equipo.py con una clase Equipo que tenga nombre, tipo, ubicacion y un método `__str__()` para mostrar una descripción legible.
2. Crear un script src/servicio_validado.py con una clase Servicio que use `@property` para normalizar el nombre y validar que el puerto esté entre 1 y 65535.
3. Crear un script src/servidor_con_interfaz.py con una clase InterfazRed y una clase Servidor que la use por composición. El servidor debe mostrar hostname, interfaz e IP.
4. Crear un script src/servicios_hereditados.py con una clase base Servicio y dos clases hijas ServicioWeb y ServicioBD, cada una con su propio método `descripcion()`.
5. Crear un script src/reporte_servidor.py que combine varios objetos: un servidor, una interfaz de red y dos servicios. Debe generar un resumen final usando f-strings y métodos de los objetos.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar source <code>.venv/bin/activate</code> desde <code>~/Curso_Python/Tema04</code> .
NameError al crear una instancia	La clase no se ha definido antes de usarla o el nombre no coincide.	Revisar el orden del código y comprobar mayúsculas, minúsculas y PascalCase.
TypeError: falta self o sobran argumentos	La firma del método no coincide con la llamada.	Asegurarse de que los métodos de instancia reciben self como primer parámetro y de que <code>__init__</code> recibe los argumentos usados al crear el objeto.
AttributeError al acceder a un atributo	El atributo no existe, no se inicializó en <code>__init__()</code> o se está usando un nombre incorrecto.	Inicializar los atributos importantes en <code>__init__()</code> y revisar nombres con guion bajo, doble guion bajo y mayúsculas.
Salida poco legible al imprimir un objeto	La clase no define <code>__str__()</code> .	Agregar un método <code>__str__()</code> que devuelva una cadena con return.

Validación de puerto falla con ValueError	Se ha asignado un puerto fuera del rango permitido.	Usar valores entre 1 y 65535 o revisar la lógica del setter.
La herencia complica el ejemplo	Se ha usado herencia para una relación que realmente era composición.	Revisar si la relación es “es un tipo de” o “tiene un”. Si es “tiene un”, usar composición.